Worksheet 10 — API Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 10 **Aligned:** OWASP API Security Top 10:2023 — API1 BOLA · API3 Broken Object Property Level Auth (Mass Assignment) · API4 Unrestricted Resource Consumption **Signature game:** 🥷 crAPI Raid
>
> **Ethics note:** Run exploits **only** against the lab targets shipped here (`vulnerable_api.py` on `:8080`) or your own OWASP crAPI instance. Never test BOLA, mass assignment, or brute-force against systems you do not own or are not explicitly authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group	AI use
Andrew Dicesare	6631503050	10/22/2026	Let me eat	AI assistant used as a working aid only — full disclosure in the note below this table

AI use, in full: I used an AI assistant to help phrase the **Audit the AI** and **Prompt Problem** answers (which are graded on *my* critique of the AI and therefore quote its output) and to review the finished worksheet. I ran the lab myself against the live API and captured every screenshot; the test choices, the reading of every status code, and the conclusions in Parts 2, 4 and the Evidence section are mine. I read each capture as it came back, rejected two of them — a `200` response that showed no body, and an `id` that did not follow from the previous task — and had them re-run before this sheet was finalised. Nothing on this sheet was accepted without being checked against the running API.

Part 2 — Lecture Questions

1. Define **BOLA (API1:2023)**. Why is it the #1 API risk, and why can't a WAF reliably stop it?

   A: BOLA (Broken Object Level Authorization) is when an endpoint takes an object identifier from the client (e.g. `/api/users/<id>/orders`) and returns the object without checking the caller is allowed to access *that specific object*. It's the #1 API risk because object IDs are exposed everywhere in REST APIs and are trivially enumerated, so one missing per-object check leaks every user's data at scale. A WAF can't reliably stop it because the requests are perfectly valid HTTP with no malicious payload — asking for `id=3` instead of `id=1` looks identical to a legitimate request; only the server knows who owns object 3, so only server-side authorization can decide.

2. What is **mass assignment / broken object property level authorization (API3:2023)**? Contrast a blocklist vs. an allow-list of bindable fields.

   A: Mass assignment is when the server binds the entire client JSON body onto an internal object, letting a client set fields it should never control — like `is_admin` or `balance` in `POST /api/users`. A **blocklist** tries to strip known-dangerous keys, but it fails open: forget one field (or add a new sensitive field later) and it's silently bindable again. An **allow-list** (`ALLOWED_CREATE_FIELDS = {"username","password"}`) only ever binds the keys you explicitly name and the server sets every sensitive field itself — it fails closed, so a smuggled `is_admin:true` is simply ignored.

3. **API4:2023** is unrestricted resource consumption. Give two distinct impacts (security and availability) of an unthrottled `/api/login`.

   A: *Security:* with no throttle or lockout an attacker can brute-force / credential-stuff passwords indefinitely — unlimited guesses eventually crack weak credentials (in this lab the loop lands `alice123`). *Availability:* each unthrottled request consumes CPU/DB work, so an attacker (or a stampede) can flood the endpoint and exhaust server resources, degrading or denying service for everyone — a cheap DoS. Rate limiting addresses both at once.

4. In `vulnerable_api.py`, `current_user()` trusts the `X-User-Id` header. Why is a client-supplied header NOT authentication, and what should replace it?

   A: The `X-User-Id` header is attacker-controlled — the client simply asserts "I am user 1" and the server believes it, with nothing proving the claim, so anyone can impersonate anyone by changing a header value. Real authentication must carry *verifiable proof* the server can validate: a signed, expiring token (a properly verified JWT with a pinned algorithm and strong secret) or a server-side session the client can't forge. Identity must be derived from something the server cryptographically checks, never from a plaintext header the client chooses.

5. Why does object-level authorization have to be checked **per request, per object** rather than once at login?

   A: Login only establishes *who* you are (authentication); it says nothing about *which* objects you may touch. Authorization is a property of the (caller, object) pair, and that relationship differs for every object the request can reach — being logged in as alice doesn't make carol's order id yours. Since the client supplies a fresh object id on each request, the ownership/permission check must run on every request against that specific object; a one-time check at login would leave every other object unguarded (exactly the BOLA in Task 1).

Part 3 — Hands-on Lab (145 min)

Learning goals: Exploit BOLA, mass assignment, and a missing rate limit against a live API; then read the secure version and explain each fix.

Prerequisites: Docker + Docker Compose, `curl`. (Optional bonus: cloned OWASP crAPI.)

Environment setup

```
cd labs/week10-api-security
docker compose up         # INSECURE API on :8080, SECURE API on :8081
# Seeded users: alice(id 1) bob(id 2) carol(id 3, admin, balance 9999)
```

What to submit per task: the exact command(s) + raw response/HTTP status, a screenshot, and a 2–3 sentence mitigation note mapping the bug to its OWASP API id.

Task 0 — Onboarding (15 min)

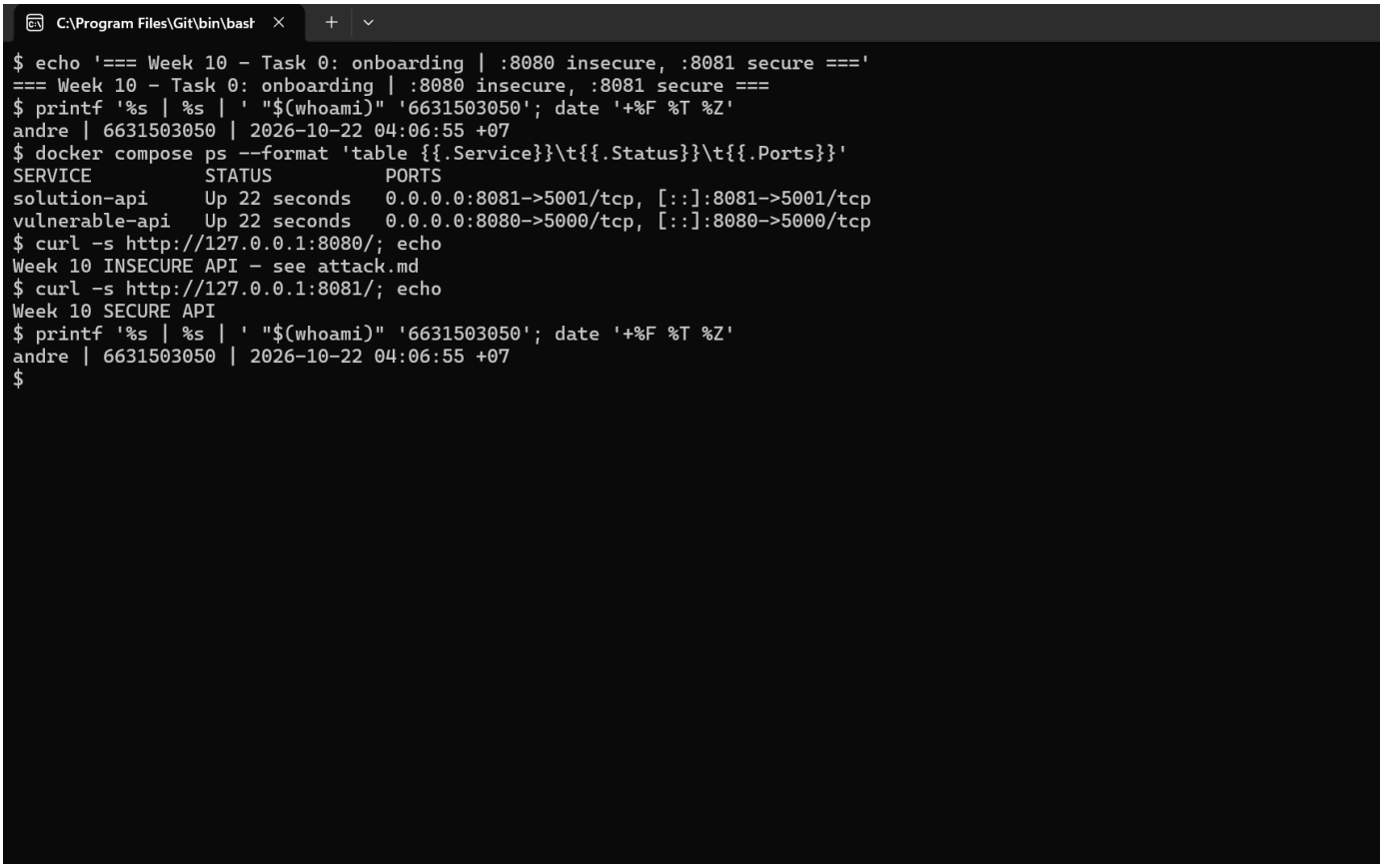
Confirm both APIs respond: `curl http://localhost:8080/` and `curl http://localhost:8081/`. Note which port is insecure. Record the three seeded users. **Deliverable:** both root responses + the user table.

Root responses:

- `:8080` → `Week 10 INSECURE API — see attack.md` — **this is the insecure target.**
- `:8081` → `Week 10 SECURE API` — the hardened reference (`solution_api.py`).

Seeded users:

id	username	password	balance	is_admin
1	alice	alice123	100	false
2	bob	bob123	50	false
3	carol	carol123	9999	true



Task 1 — BOLA: read another user's orders (35 min)

Goal: Read carol's (admin) orders without being carol — API1:2023. **Steps:**

1. `curl http://localhost:8080/api/users/3/orders` — note you receive the "Server rack" order with no auth.
2. Iterate the id: `curl http://localhost:8080/api/users/2/orders`.
3. On the secure API observe the ladder: `curl -i http://localhost:8081/api/users/3/orders` (401), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders` (403), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders` (200). **Deliverable:** the leaked order JSON + the 401/403/200 transcript + mitigation note.

Leaked JSON (insecure :8080, no auth at all):

```
GET /api/users/3/orders  ->  [{"item":"Server rack","order_id":"C-1","total":5000},
                              {"item":"FLAG{bola_demo}","order_id":"C-FLAG","total":0}]
GET /api/users/2/orders  ->  [{"item":"Mouse","order_id":"B-1","total":15}]
```

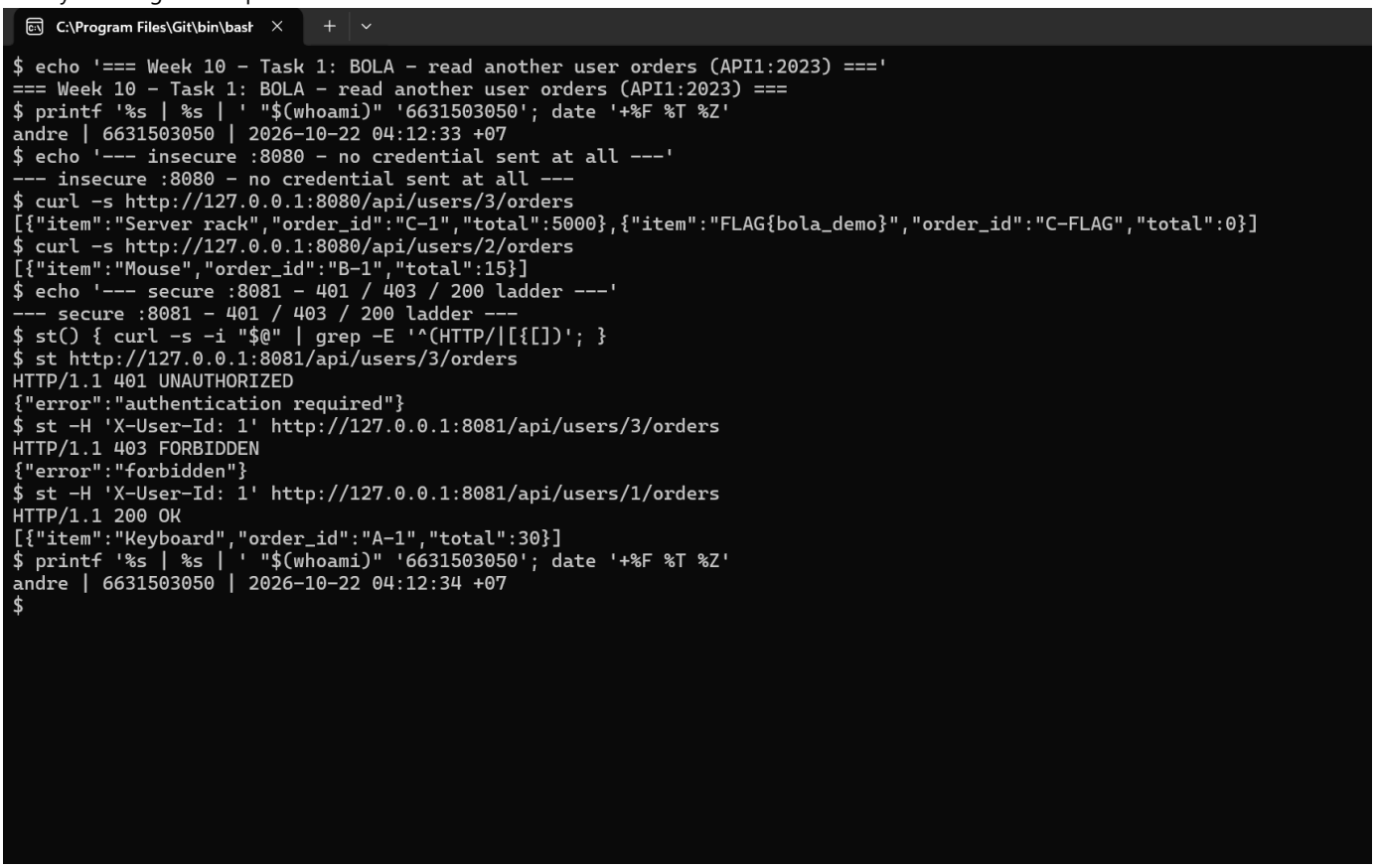
Changing the id in the URL returns any user's orders — carol's admin order (and its flag) leak to an unauthenticated caller.

Secure :8081 ladder:

```
GET /api/users/3/orders                 -> 401 {"error":"authentication required"}   (no identity)
GET /api/users/3/orders  X-User-Id: 1   -> 403 {"error":"forbidden"}                 (alice may not
read carol's)
GET /api/users/1/orders  X-User-Id: 1   -> 200 [{"item":"Keyboard","order_id":"A-1","total":30}]  (own
orders OK)
```

Mitigation (API1:2023 BOLA): add a per-object ownership check before returning the object — `if caller["id"] != uid and not caller["is_admin"]: return 403`. Authorization must be evaluated per request against the specific object id, never inferred from

merely reaching the endpoint.



Task 2 — Mass assignment: self-promote to admin (30 min)

Goal: Smuggle `is_admin` + `balance` into user creation — API3:2023. **Steps:**

1. `curl -X POST http://localhost:8080/api/users -H "Content-Type: application/json" -d '{"username":"mallory","password":"x","is_admin":true,"balance":1000000}'`
2. Confirm the response echoes `"is_admin": true, "balance": 1000000`.
3. Repeat against `:8081` and confirm the smuggled fields are forced to `is_admin:false, balance:0`. **Deliverable:** both responses side by side + mitigation note (allow-list in `solution_api.py`).

Insecure :8080 (smuggled fields bind — and the admin flag drops):

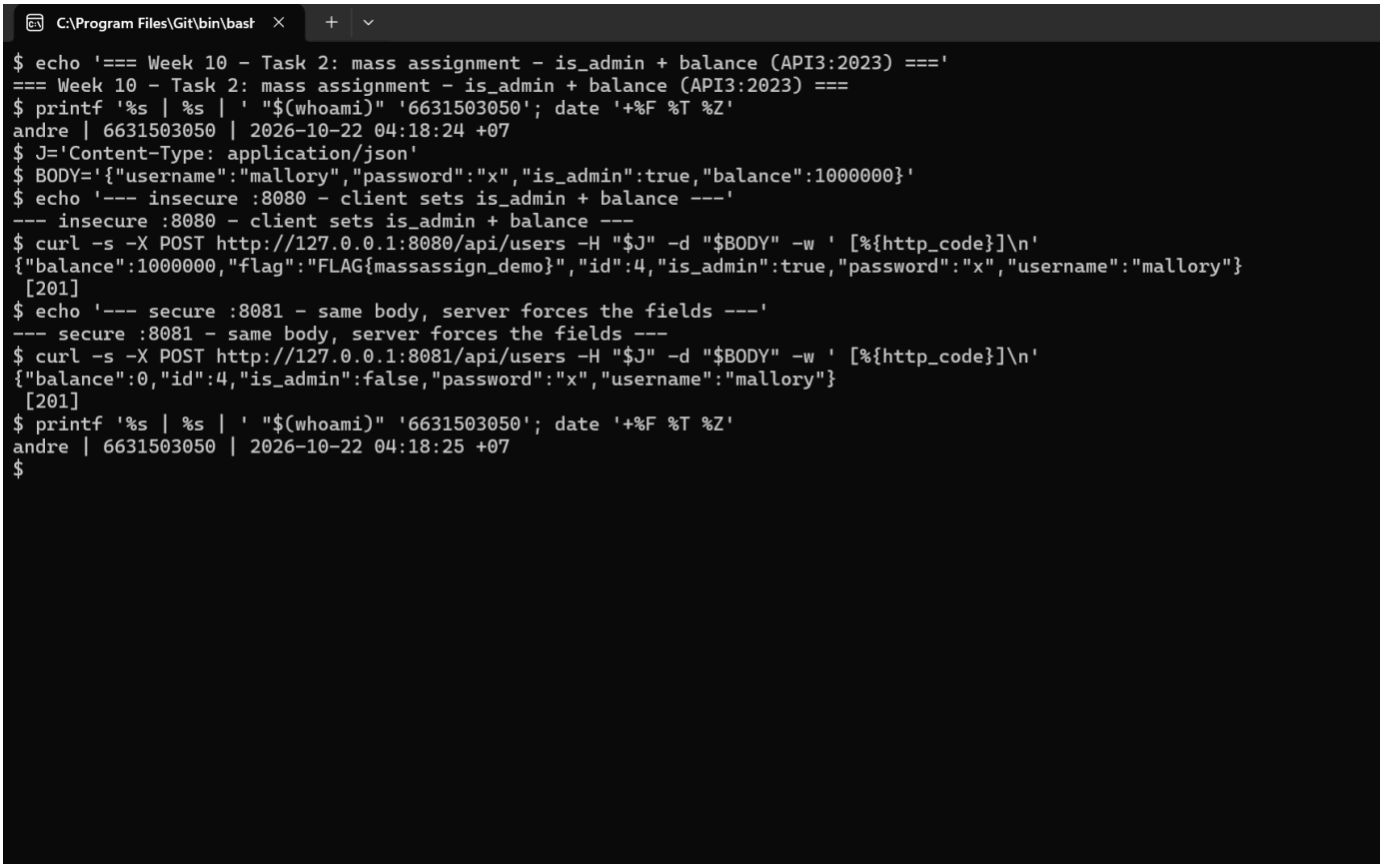

```
{"balance":1000000,"flag":"FLAG{massassign_demo}","id":4,"is_admin":true,"password":"x","username":"mallory"}
```


Secure :8081 (server forces sensitive fields):


```
{"balance":0,"id":4,"is_admin":false,"password":"x","username":"mallory"}
```


The insecure handler does `user.update(body)`, trusting every client key; the secure handler binds only `ALLOWED_CREATE_FIELDS = {"username","password"}` and sets `id/balance/is_admin` itself, so `is_admin:true` and `balance:1000000` are ignored.

Mitigation (API3:2023 mass assignment): allow-list the fields a client may set; the server owns all privileged/sensitive properties. Fail closed — an unknown or sensitive key is dropped, not bound.



Task 3 — Unrestricted resource consumption: brute-force login (25 min)

Goal: Show `/api/login` has no throttle — API4:2023. **Steps:**

1. Run the brute-force loop from `attack.md` against `:8080` (guesses incl. `alice123`) — all attempts processed.
2. On `:8081` loop 7 times: `for i in $(seq 1 7); do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8081/api/login -H "Content-Type: application/json" -d '{"username":"alice","password":"wrong"}'; done`
3. Confirm the 6th/7th attempt returns `429`. **Deliverable:** the `401 401 401 401 401 429 429` sequence + mitigation note.

Insecure :8080 (every guess processed, no throttle):

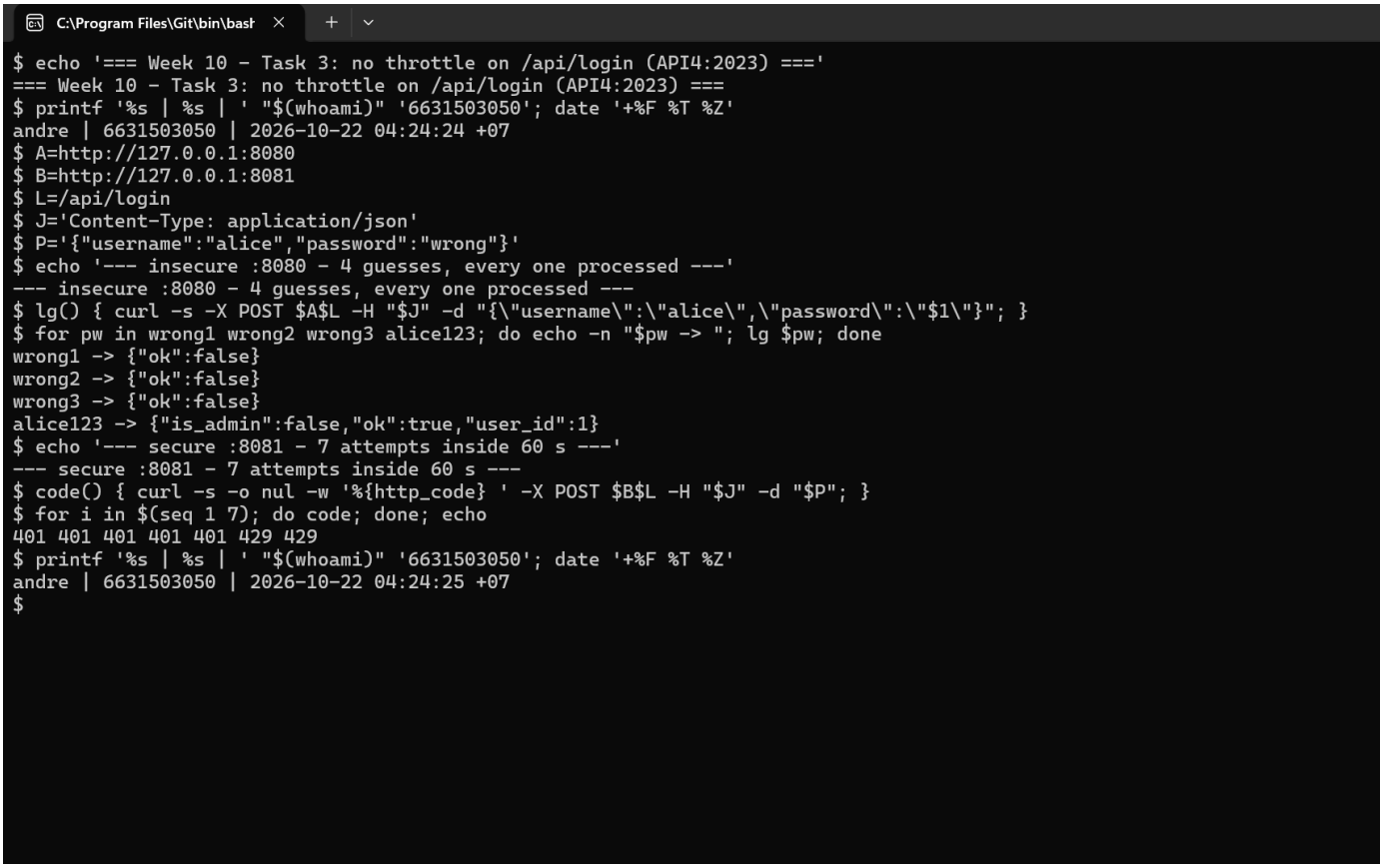
```
wrong1   -> {"ok":false}
wrong2   -> {"ok":false}
wrong3   -> {"ok":false}
alice123 -> {"is_admin":false,"ok":true,"user_id":1}   <-- password cracked, unlimited attempts
```

Secure :8081 (7 attempts within 60s):

```
401 401 401 401 401 429 429
```

The 6th and 7th attempts are rejected with `429 Too Many Requests` by the per-IP limiter (`RATE_LIMIT=5 / RATE_WINDOW=60`).

Mitigation (API4:2023 unrestricted resource consumption): enforce a per-IP (and per-account) rate limit / lockout plus exponential backoff on auth endpoints, so brute-force and resource-exhaustion both stop at the threshold.



Task 4 — Bonus: crAPI Raid (optional, 20 min)

Against your own OWASP crAPI instance (`git clone https://github.com/OWASP/crAPI.git`), capture one BOLA or mass-assignment flag. **Deliverable:** flag + endpoint + which API Top 10 id it maps to.

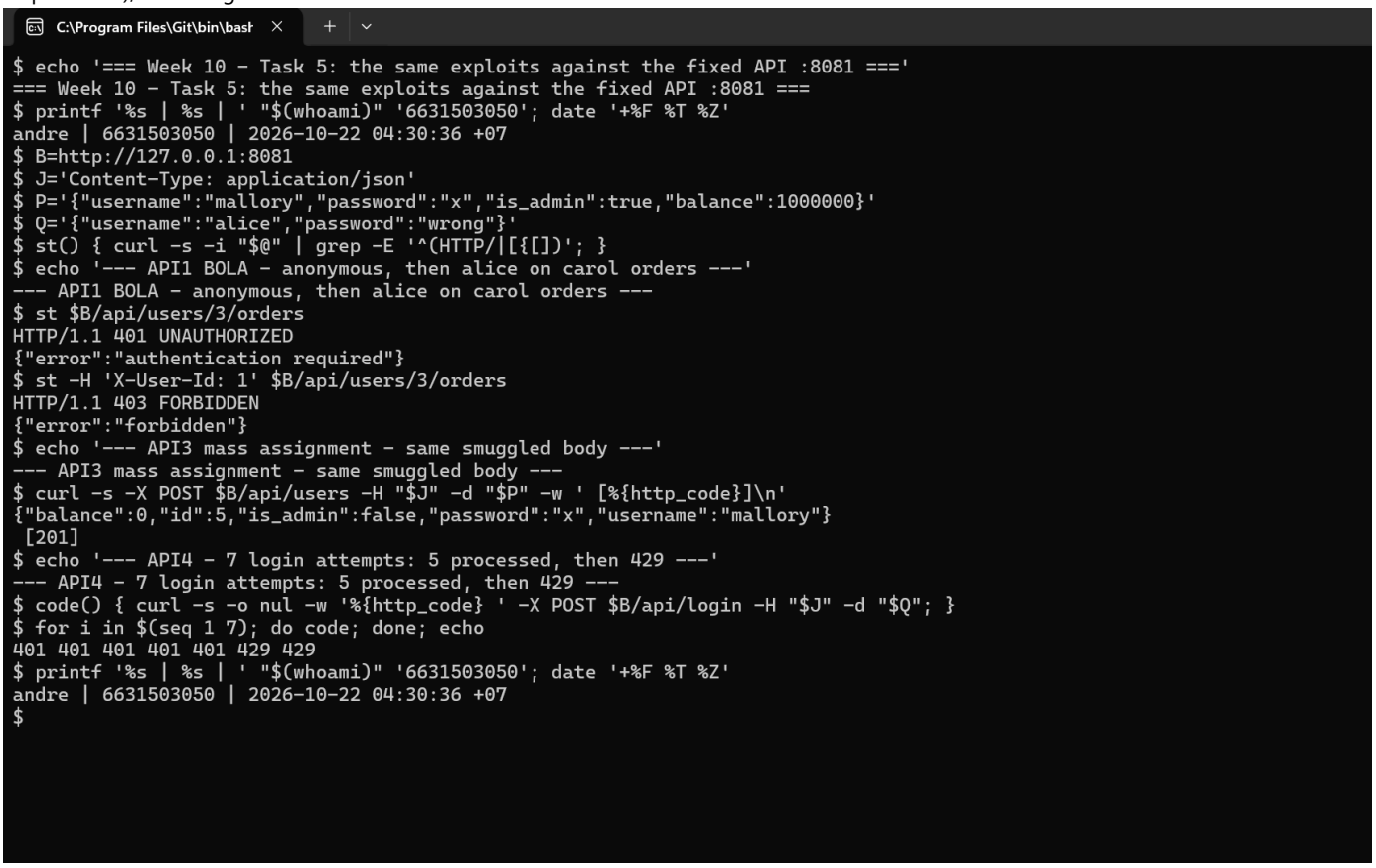
A: Optional/ungraded — not attempted in this submission. The graded target is the local lab API above (per the README: "Target: the local API below — that's the graded lab, not crAPI"). The two local flags captured in Tasks 1–2 (`FLAG{bola_demo}` via BOLA on `/api/users/3/orders`, API1; `FLAG{massassign_demo}` via mass assignment on `POST /api/users`, API3) are the graded evidence.

Task 5 — Defend / fix it (20 min)

Goal: Explain the fixes using the secure reference `solution_api.py`. **Steps:** Read the three `# --- FIX ...` blocks. For each, quote the line(s) that defeat your Task 1–3 exploit: the per-object ownership check (`caller["id"] != uid and not caller["is_admin"]`), `ALLOWED_CREATE_FIELDS` binding, and the `RATE_LIMIT=5 / RATE_WINDOW=60` limiter. **Deliverable:** for each of API1/API3/API4 — the exploit that worked on `:8080`, the line in `solution_api.py` that blocks it, and the new HTTP status on `:8081`.

OWASP id	Exploit that worked on `:8080`	Fix line in `solution_api.py`	New status on `:8081`
API1 BOLA	`GET /api/users/3/orders` leaked carol's orders with no auth	`if caller is None: return 401` then `if caller["id"] != uid and not caller["is_admin"]: return 403` (object-level ownership check)	**401** (no id) / **403** (not owner) / **200** (own)
API3 Mass assignment	`POST /api/users` with `is_admin:true, balance:1000000` bound those fields	`ALLOWED_CREATE_FIELDS = {"username","password"}` + `user = {k: body[k] for k in ALLOWED_CREATE_FIELDS if k in body}` then `user.update({"id":_next_id,"balance":0,"is_admin":False})`	**201** with `is_admin:false, balance:0` (smuggled fields ignored)
API4 Resource consumption	Unlimited `/api/login` guesses cracked `alice123`	`RATE_LIMIT = 5 / RATE_WINDOW = 60` + `if rate_limited(request.remote_addr): return 429`	**429** on the 6th attempt within 60s

Why they work: each fix moves the decision server-side and fails closed — the server checks ownership on every object, binds only safe fields and sets the rest itself, and caps attempts per IP. The exploits depend on the server trusting the client (its id, its full body, its request rate); removing that trust defeats all three.



Part 4 — Reflection

1. **Mapping:** complete a table — finding | endpoint | OWASP API id | fix applied.

   A:

Finding	Endpoint	OWASP API id	Fix applied
BOLA — reads any user's orders by id	`GET /api/users/<id>/orders`	**API1:2023** Broken Object Level Authorization	Per-object ownership check (401 if no identity, 403 unless owner/admin)
Mass assignment — client sets `is_admin`/`balance`	`POST /api/users`	**API3:2023** Broken Object Property Level Authorization	Allow-list bindable fields; server sets sensitive fields
No rate limit — unlimited login guesses	`POST /api/login`	**API4:2023** Unrestricted Resource Consumption	Per-IP limiter (5 / 60s → 429)
Spoofable identity — trusts `X-User-Id` header	all authenticated routes	Underlies API1 (no real authentication)	Replace header trust with a verified token/session

2. **Real breach:** research an API authorization breach (e.g., the Optus or T-Mobile BOLA/IDOR incidents). Which API Top 10 id matches, and would the `solution_api.py` ownership check have prevented it?

   A: The **2022 Optus breach** exposed millions of customer records through an API endpoint where customer identifiers could be enumerated with weak/absent authorization — a textbook **API1:2023 BOLA/IDOR**. It maps directly to Task 1: walking sequential ids returns records that aren't yours because the server never checks ownership per object. The `solution_api.py` ownership check (`if caller["id"] != uid and not caller["is_admin"]: return 403`) would have prevented the bulk extraction, because each request for another customer's id would have been rejected with 403 instead of returning data — authentication/reachability alone would no longer grant access.

3. **Best mitigation:** of object-level auth, allow-list binding, and rate limiting — which gives the most risk reduction for an API platform, and why?

   A: Object-level authorization gives the most risk reduction. BOLA is the most common and highest-impact API flaw because object ids are exposed on nearly every endpoint and enumeration turns one flaw into bulk data exfiltration of *all* users. Allow-list binding and rate limiting are essential but narrower — they protect specific operations (creation, auth) — whereas a per-object ownership check guards every read/write of every resource and is the control that fails safe even when a token is stolen or over-privileged. It's the backbone; the other two are important supporting layers.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–4: commands, output, screenshots)	40
Defense (Task 5: fixes mapped to `solution_api.py`)	25
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted. *Each capture below opens and closes with that command typed into the same terminal and its output — `andre | 6631503050 | 2026-10-22 04:06:55 +07`, `… 04:12:33 +07`, `… 04:18:24 +07`, `… 04:24:24 +07`, `… 04:30:36 +07` — so the command, the time and the evidence share one image. All five ran against one uninterrupted stack: Task 0 shows it 22 s old and no `docker compose restart` occurs between captures, which is why the auto-increment ids in Task 2 (`id:4`) and Task 5 (`id:5`) line up. The stamp's `whoami` reads `andre` — my Windows account name, a shortened form of Andrew Dicesare; the student ID 6631503050 matches Part 1.*
- **Personalized flag (if this lab issues one):** `FLAG{bola_demo}` (captured in carol's `C-FLAG` order via the Task 1 BOLA) and `FLAG{massassign_demo}` (returned by `POST /api/users` when `is_admin:true` is smuggled in Task 2). This instance was unseeded, so it returns the default demo flags rather than per-student ones. *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*

  1. What did you do, and **why did the vulnerability work**?

     A: I read carol's admin orders just by changing the id in `/api/users/<id>/orders`, created a user that smuggled `is_admin:true` and a million-dollar balance into `POST /api/users`, and brute-forced `/api/login` until it accepted `alice123`. They worked because the server trusted the client at every step: no per-object ownership check (so any id returns that user's data), it bound the whole request body (so I set privileged fields), and it never throttled login (so I could guess forever).

  2. **Why does your fix actually stop it** — and what could still break it?

     A: The secure version checks ownership per object (401/403 unless you own it or are admin), binds only an allow-list and sets sensitive fields server-side, and rate-limits login per IP (429 after 5 tries). Forged ids, smuggled fields, and brute-force all fail because the decision is now server-side and fails closed. What could still break it: the `X-User-Id` header is still not real authentication (it should be a verified token), the rate limiter is per-IP so a botnet/proxy rotation can sidestep it, and a *new* endpoint that forgets the ownership check reopens BOLA — authorization must be applied on every object access, not just these routes.

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

   AI answer (fix for the BOLA endpoint):

   ```
   @app.get("/api/users/<int:uid>/orders")
   def get_orders(uid):
       caller = current_user()
       # make sure the user is logged in before returning data
       if caller is None:
           return jsonify({"error": "authentication required"}), 401
       return jsonify(ORDERS.get(uid, []))
   ```

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

   A: The fix adds authentication but **not authorization** — it only checks `if caller is None`, then does `return jsonify(ORDERS.get(uid, []))` for *any* `uid`. This is the classic BOLA mistake: it confirms you're logged in but never checks the caller owns `uid`, so alice (id 1) can still read carol's (id 3) orders — the exact Task 1 exploit still succeeds. It conflates authentication ("are you someone?") with object-level authorization ("may you read *this* object?").

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

   Correct version:

   ```
   @app.get("/api/users/<int:uid>/orders")
   def get_orders(uid):
       caller = current_user()
       if caller is None:
           return jsonify({"error": "authentication required"}), 401
       # object-level authorization: own orders only, unless admin
       if caller["id"] != uid and not caller["is_admin"]:
           return jsonify({"error": "forbidden"}), 403
       return jsonify(ORDERS.get(uid, []))
   ```

   Verified: with `X-User-Id: 1`, `/api/users/3/orders` → **403** (was leaking carol's orders), while `/api/users/1/orders` → **200** with alice's own order. **Why the AI was insufficient:** authentication without a per-object ownership check leaves BOLA fully open — the data still leaks by changing the id. The real fix compares the caller to the requested object and denies by default (403) unless they own it or are an admin.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

A: The API lets you ask for a user's orders by putting their number in the URL, lets you create an account by sending a blob of JSON, and lets you try to log in as many times as you want. It's exploitable because it trusts whatever the client sends: it never checks the orders you're asking for are *yours* (so you read other people's by changing the number), it copies every field you send into the new account (so you make yourself an admin), and it never limits login tries (so you can guess passwords forever). In short, it checks almost nothing on the server side and believes the client about identity, fields, and intent.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

Final prompt: "In this Flask route `GET /api/users/<int:uid>/orders`, fix the BOLA (API1:2023). The caller's identity comes from `current_user()`. Return 401 if there is no authenticated caller, then return 403 unless `caller['id'] == uid` OR `caller['is_admin']` is true; only then return the orders. Deny by default. Return only the fixed function." **Verified result:** The AI produced the function with both guards (`401` when `caller is None`, `403` when `caller["id"] != uid and not`

`caller["is_admin"]`). Re-firing `GET /api/users/3/orders` with `X-User-Id: 1` now returns **403 forbidden** instead of carol's orders, while `GET /api/users/1/orders` with `X-User-Id: 1` still returns **200** — exploit fails, legitimate access preserved. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*